Line-by-Line Code Explanation: Student Record Management System

This document walks through `student_records.py` from top to bottom. Each section shows the code, followed by a plain-English explanation of what every line does and why it is there.

Big picture: the program has four layers.

1. **Data model** – the `Student` class.
2. **Validation** – functions that clean and check user input.
3. **Storage** – the `StudentRepository` class that talks to SQLite.
4. **User interface** – the menu and the functions behind each menu option.

---

1. Module Docstring

```
"""Student Record Management System -
single file version.
Run:  python student_records.py
```

```
Data is saved in students.db (created
automatically next to this file).
"""
```

- A triple-quoted description of the program. It does not affect how the code runs.
- It tells the user how to run the file and where the data is saved.

---

2. Imports

```
import math
import re
import sqlite3
import time
from contextlib import contextmanager
from dataclasses import dataclass
from pathlib import Path
```

- `math` – used for `math.isfinite()` to reject `nan` and `inf` as GPA values.
- `re` – regular expressions, used to check email and roll number formats.
- `sqlite3` – Python's built-in SQLite database module.
- `time` – used to create a timestamp for the corrupt-database backup name.

- `contextmanager` – a decorator that turns a function into a `with` statement helper.
- `dataclass` – a decorator that creates simple data classes with minimal code.
- `Path` – a clean, cross-platform way to work with file paths.

---

3. Constants

```
DB_FILE = Path(__file__).parent /
"students.db"
MAX_GPA = 10.0
```

- `__file__` is the path of the script itself. `.parent` is the folder containing it. `/ "students.db"` appends the file name. So the database always lives **next to the script**, no matter where you run it from.
- `MAX_GPA` is the highest allowed GPA. Changing this one value (for example to `4.0`) changes the rule everywhere.

---

4. The `Student` Data Class

```
@dataclass
class Student:
```

```
    roll_no: str
    name: str
    email: str
    gpa: float
    course: str
```

- `@dataclass` automatically writes the `__init__` method, so `Student("24BIB001", "Asha", "a@b.com", 8.5, "BCA")` just works.
- It is a simple container for one student's five fields, with the expected type of each.

---

5. Validation

The validation exception

```
class ValidationError(ValueError):
    pass
```

- A custom error raised when user input is invalid.
- It inherits from `ValueError`, which makes sense because the *value* is wrong. `pass` means it adds nothing new.

Regular expressions

```
EMAIL_RE = re.compile(r"^[A-Za-z0-
9._%+-]+@[A-Za-z0-9-]+(\.[A-Za-z0-9-]+)*\.
[A-Za-z]{2,}$")
ROLL_RE = re.compile(r"^[A-Z0-9][A-Z0-9-]
{2,14}$")
```

- `re.compile` prepares a pattern once so it can be reused quickly.
- EMAIL_RE breakdown:
  - `^` and `$` – the pattern must match the whole string.
  - `[A-Za-z0-9._%+-]+` – the part before `@`: one or more letters, digits, or the symbols `. _ % + -`.
  - `@` – the literal at sign.
  - `[A-Za-z0-9-]+` – the first domain label (for example `example`).
  - `(\.[A-Za-z0-9-]+)*` – zero or more extra labels (for example `.co`).
  - `\.[A-Za-z]{2,}` – a final dot plus a top-level domain of at least 2 letters (`.com`, `.in`).
- ROLL_RE breakdown:
  - `[A-Z0-9]` – the first character must be an uppercase letter or digit (so it cannot start with a hyphen).

    - `[A-Z0-9-]{2,14}` – then 2 to 14 more characters: uppercase letters, digits, or hyphens.
    - Total length: 3 to 15 characters.

`validate_roll_no`

```
def validate_roll_no(raw):
    value = raw.strip().upper()
    if not ROLL_RE.match(value):
        raise ValidationError("Roll number
must be 3-15 characters: letters, digits or
hyphens (e.g. 24BIB001).")
    return value
```

- `raw.strip()` removes spaces at both ends; `.upper()` converts to uppercase, so `24bib001` becomes `24BIB001`.
- `ROLL_RE.match(value)` checks the pattern. If it does not match, a `ValidationError` is raised with a helpful message.
- If valid, the cleaned value is returned.

`validate_name`

```
def validate_name(raw):
    value = " ".join(raw.split())
    if not 2 <= len(value) <= 60:
        raise ValidationError("Name must be
2-60 characters long.")
```

```
    if not all(c.isalpha() or c in " .'-"
for c in value) or sum(c.isalpha() for c in
value) < 2:
        raise ValidationError("Name may
contain only letters, spaces, . ' and -.")
    return value
```

- `raw.split()` splits on any whitespace and drops empty pieces; `" ".join(...)` glues them back with single spaces. So `" Asha Rani "` becomes `"Asha Rani"`.
- The first check enforces a length of 2 to 60 characters (Python allows chained comparisons like `2 <= x <= 60`).
- The second check:
  - `all(c.isalpha() or c in " .'-" for c in value)` – every character must be a letter or one of: space, dot, apostrophe, hyphen.
  - `sum(c.isalpha() for c in value) < 2` – counts the letters (each `True` counts as 1); a name needs at least 2 letters, so names like `". -"` are rejected.
- Returns the cleaned name.

`validate_email`

```
def validate_email(raw):
    value = raw.strip().lower()
    if len(value) > 254 or not
```

```
EMAIL_RE.match(value):
        raise ValidationError("Enter a
valid email address (e.g.
name@example.com).")
    return value
```

- Trims spaces and converts to lowercase.
- 254 characters is the maximum length of an email address by standard.
- Rejects the value if it is too long or does not match the email pattern.

`validate_gpa`

```
def validate_gpa(raw):
    try:
        value = float(raw.strip())
    except ValueError:
        raise ValidationError("GPA must be
a number (e.g. 8.5).") from None
```

- Tries to convert the text to a decimal number.
- If that fails (for example `"abc"`), a `ValidationError` is raised instead. `from None` hides the original `ValueError` so the user only sees the clean message.

```
    if not math.isfinite(value) or not 0.0
<= value <= MAX_GPA:
        raise ValidationError(f"GPA must be
```

```
between 0 and {MAX_GPA:g}.")
    return round(value, 2)
```

- `math.isfinite` is `False` for `nan` and `inf`, which Python's `float()` would otherwise accept.
- The value must be between 0 and `MAX_GPA`.
- `{MAX_GPA:g}` prints `10` instead of `10.0`.
- `round(value, 2)` keeps two decimal places (`8.456` becomes `8.46`).

`validate_course`

```
def validate_course(raw):
    value = " ".join(raw.split())
    if not 2 <= len(value) <= 50:
        raise ValidationError("Course must
be 2-50 characters long.")
    return value
```

- Collapses extra spaces, then checks the length is between 2 and 50.

---

6. Storage Exceptions

```
class StorageError(Exception):
    pass
```

```
class DuplicateStudentError(StorageError):
    pass


class StudentNotFoundError(StorageError):
    pass


class CorruptDatabaseError(StorageError):
    pass
```

- `StorageError` is the parent for all database-related problems.
- The three child classes describe specific problems: a duplicate roll number, a missing student, and a damaged database file.
- Because they share a parent, the program can catch **all** storage problems with one `except StorageError`, or catch a specific one when needed.

---

7. Database Schema

```
SCHEMA = """
CREATE TABLE IF NOT EXISTS students (
    roll_no TEXT PRIMARY KEY,
    name    TEXT NOT NULL,
    email   TEXT NOT NULL,
```

```
    gpa     REAL NOT NULL CHECK (gpa >= 0),
    course  TEXT NOT NULL
)
"""
```

- `CREATE TABLE IF NOT EXISTS` creates the table only if it does not already exist.
- `roll_no TEXT PRIMARY KEY` – each roll number must be unique; this is how duplicates are blocked.
- `NOT NULL` – the field can never be empty.
- `CHECK (gpa >= 0)` – the database itself refuses negative GPAs, as an extra safety net on top of Python validation.

---

8. The `StudentRepository` Class

This class is the only part of the program that touches the database. Everything else calls its methods.

Constructor

```
class StudentRepository:
    def __init__(self, db_path):
        self.db_path = Path(db_path)
        self._init_db()
```

- Stores the database path (converted to a `Path`).
- Immediately calls `_init_db()` to check and prepare the database.

The `_session` context manager

```
    @contextmanager
    def _session(self):
        conn = None
        try:
            conn =
sqlite3.connect(self.db_path, timeout=5)
            conn.row_factory = sqlite3.Row
            yield conn
            conn.commit()
```

- `@contextmanager` lets this method be used as `with self._session() as conn:`.
- `conn = None` first, so the `finally` block below can safely check whether a connection was made.
- `sqlite3.connect(..., timeout=5)` opens the database; if another program has it locked, it waits up to 5 seconds.
- `row_factory = sqlite3.Row` makes rows accessible by column name (`row["name"]`) instead of position.

- `yield conn` hands the connection to the code inside the `with` block. When that block finishes without errors, execution continues to `conn.commit()`, which saves the changes.

```
        except StorageError:
            if conn:
                conn.rollback()
            raise
```

- If the code inside the `with` block raises one of our own `StorageError`s (such as a duplicate or not found), the changes are **rolled back** (undone) and the error is re-raised unchanged.

```
        except sqlite3.OperationalError as
exc:
            raise StorageError(f"Database
error: {exc}") from exc
        except sqlite3.DatabaseError as
exc:
            raise
CorruptDatabaseError(f"Database file is
corrupted: {exc}") from exc
```

- `OperationalError` (for example, database locked or table missing) is converted into our `StorageError`.
- Any other `DatabaseError` is treated as corruption.

- **Order matters:** `OperationalError` is a *subclass* of `DatabaseError`, so the more specific one must come first, otherwise it would never be reached.
- `from exc` keeps the original error attached for debugging.

```
        finally:
            if conn:
                conn.close()
```

- `finally` always runs, whether or not an error happened. It makes sure the connection is closed so the file is never left locked.

`_init_db`

```
    def _init_db(self):
        with self._session() as conn:
            result = conn.execute("PRAGMA
quick_check").fetchone()
            if result is None or result[0]
!= "ok":
                raise
CorruptDatabaseError("Database integrity
check failed.")
            conn.execute(SCHEMA)
            conn.execute("SELECT roll_no,
name, email, gpa, course FROM students
LIMIT 1")
```

- Opens a session and runs SQLite's `PRAGMA quick_check`, a fast integrity test. A healthy database returns `"ok"`.
- If the result is anything else, `CorruptDatabaseError` is raised.
- `conn.execute(SCHEMA)` creates the table if it is missing.
- The final `SELECT ... LIMIT 1` confirms the table really has the expected columns. If an old or different table is present, this fails and is reported as a storage error.

`reset_corrupt_database`

```
    def reset_corrupt_database(self):
        """Move the bad file aside (never
deletes it) and start a fresh database."""
        backup = self.db_path.with_name(f"
{self.db_path.name}.corrupt-
{int(time.time())}")
        self.db_path.replace(backup)
        self._init_db()
        return backup
```

- Builds a backup name like `students.db.corrupt-1760500000`, using the current time in seconds so each backup is unique.

- `self.db_path.replace(backup)` renames the damaged file to the backup name. Nothing is deleted, so data can still be recovered manually.
- `_init_db()` then creates a brand new database at the original path.
- Returns the backup path so the caller can show its name.

`_row`

```
    @staticmethod
    def _row(row):
        return Student(row["roll_no"],
row["name"], row["email"], row["gpa"],
row["course"])
```

- `@staticmethod` means this method does not need `self`.
- Converts one database row into a `Student` object.

`add`

```
    def add(self, s):
        with self._session() as conn:
            try:
                conn.execute("INSERT INTO
students VALUES (?, ?, ?, ?, ?)",
                             (s.roll_no,
```

```
s.name, s.email, s.gpa, s.course))
            except sqlite3.IntegrityError:
                raise
DuplicateStudentError(f"Roll number
{s.roll_no} already exists.") from None
```

- Inserts a new row. The `?` marks are placeholders: values are passed separately, which prevents SQL injection.
- If the roll number already exists, SQLite raises `IntegrityError` (primary key violation). It is caught here and converted into a friendlier `DuplicateStudentError`.
- This is caught inside the `with` block, which is why it never reaches the `DatabaseError` handler in `_session`.

`get`

```
    def get(self, roll_no):
        with self._session() as conn:
            row = conn.execute("SELECT *
FROM students WHERE roll_no = ?",
(roll_no,)).fetchone()
        if row is None:
            raise StudentNotFoundError(f"No
student with roll number {roll_no}.")
        return self._row(row)
```

- Looks up one student by roll number. `fetchone()` returns one row, or `None` if nothing matches.
- `(roll_no,)` is a one-item tuple (the trailing comma is required).
- If no row is found, it raises `StudentNotFoundError`; otherwise the row is converted to a `Student`.

`list_all`

```
    def list_all(self):
        with self._session() as conn:
            rows = conn.execute("SELECT *
FROM students ORDER BY roll_no").fetchall()
        return [self._row(r) for r in rows]
```

- Fetches every student sorted by roll number and converts each row to a `Student` using a list comprehension.

`search`

```
    def search(self, query):
        """Case-insensitive partial match
on roll number or name."""
        q = query.strip().casefold()
        if not q:
            return []
        return [s for s in self.list_all()
```

```
if q in s.roll_no.casefold() or q in
s.name.casefold()]
```

- `casefold()` is a stronger version of `lower()` for case-insensitive comparison.
- An empty search returns an empty list.
- Otherwise it loads all students and keeps those whose roll number **or** name *contains* the search text (`q in ...` is a substring test).

`update`

```
    def update(self, roll_no, **changes):
        self.get(roll_no)
        sets = ", ".join(f"{k} = ?" for k
in changes)
        with self._session() as conn:
            conn.execute(f"UPDATE students
SET {sets} WHERE roll_no = ?",
(*changes.values(), roll_no))
```

- `**changes` collects keyword arguments into a dictionary, for example `{"name": "Asha", "gpa": 9.0}`.
- `self.get(roll_no)` is called first only to confirm the student exists; it raises `StudentNotFoundError` otherwise.
- `sets` builds the text `"name = ?, gpa = ?"` from the dictionary keys. Column names come

from the program's own code (never from user input), so building them this way is safe, and the actual **values** still go through `?` placeholders.

- `(*changes.values(), roll_no)` unpacks the values and adds the roll number at the end, matching the order of the `?` marks.

`delete`

```
    def delete(self, roll_no):
        with self._session() as conn:
            cur = conn.execute("DELETE FROM
students WHERE roll_no = ?", (roll_no,))
            if cur.rowcount == 0:
                raise
StudentNotFoundError(f"No student with roll
number {roll_no}.")
```

- Deletes the matching row.
- `cur.rowcount` is the number of rows affected. If it is `0`, no such student existed, so an error is raised (which also triggers a rollback).

---

9. The Menu

```
MENU = """
==== Student Record Management ====
```

```
 1. Add student
 2. View all students
 3. Search (roll no. or name)
 4. Update student
 5. Delete student
 0. Exit
"""
```

- A multi-line string that is printed every time the loop repeats.

---

10. Helper Functions

`ask`

```
def ask(prompt, validator, keep=None):
    """Keep asking until input is valid. If
keep is given, empty Enter keeps old
value."""
    while True:
        raw = input(prompt).strip()
        if raw == "" and keep is not None:
            return keep
        try:
            return validator(raw)
        except ValidationError as exc:
            print(f"  x {exc}")
```

- A reusable input function. It takes the prompt text, a validator function, and an optional `keep`

value.

- `while True` repeats until a valid value is returned.
- If the user presses Enter and a `keep` value exists, the old value is returned (used during updates).
- It uses `keep is not None` (not just `if keep`) so that a kept GPA of `0.0` still works.
- Otherwise the validator runs. If it raises `ValidationError`, the message is printed and the loop asks again.

`print_table`

```
def print_table(students):
    if not students:
        print("  (no records)")
        return
```

- If the list is empty, prints a message and stops.

```
    header = f"{'Roll No':<12}{'Name':<24}
{'Email':<30}{'GPA':<6}Course"
    print(header)
    print("-" * (len(header) + 12))
```

- `{'Roll No':<12}` left-aligns the text in a column 12 characters wide. This creates neat columns.

- Prints the header, then a line of dashes slightly longer than the header.

```
    for s in students:
        print(f"{s.roll_no:<12}
{s.name[:22]:<24}{s.email[:28]:<30}{s.gpa:
<6.2f}{s.course}")
    print(f"\n  {len(students)} record(s)")
```

- For each student, prints one row using the same column widths.
- `s.name[:22]` and `s.email[:28]` cut long values so they cannot break the alignment.
- `{s.gpa:<6.2f}` shows the GPA with exactly two decimals.
- Finally, prints the total number of records.

---

11. Menu Actions

`add_student`

```
def add_student(repo):
    print("\n-- Add student (Ctrl+C to
cancel) --")
    roll = ask("Roll no : ",
validate_roll_no)
    try:
        repo.get(roll)
```

```
        print(f"  x Roll number {roll}
already exists.")
        return
    except StudentNotFoundError:
        pass
```

- Asks for a valid roll number first.
- Then checks whether it already exists: if `repo.get` succeeds, the student exists, so it prints a message and returns early. If `StudentNotFoundError` is raised, the roll number is free and the code continues (`pass`).
- Checking early saves the user from typing all the other details only to be rejected.

```
    name = ask("Name    : ", validate_name)
    email = ask("Email   : ",
validate_email)
    gpa = ask("GPA     : ", validate_gpa)
    course = ask("Course  : ",
validate_course)
    student = Student(roll, name, email,
gpa, course)
    repo.add(student)
    print("  OK Student added.")
    print_table([student])
```

- Collects the remaining fields, each with its own validator.

- Builds a `Student`, saves it, and displays the new record in a one-row table.

`view_all`

```
def view_all(repo):
    print()
    print_table(repo.list_all())
```

- Prints a blank line, then a table of all students.

`search_students`

```
def search_students(repo):
    q = input("\nSearch (partial roll no.
or name): ").strip()
    if not q:
        print("  x Enter something to
search for.")
        return
    print()
    print_table(repo.search(q))
```

- Asks for search text; an empty entry shows an error and returns.
- Otherwise prints the matching students.

`update_student`

```
def update_student(repo):
    roll = ask("\nRoll no to update: ",
validate_roll_no)
    cur = repo.get(roll)
    print("  Press Enter to keep the
current value.")
```

- Asks which student to update and loads the current record (raises an error if not found; this is handled in `main`).

```
    changes = {
        "name": ask(f"Name   [{cur.name}]:
", validate_name, keep=cur.name),
        "email": ask(f"Email
[{cur.email}]: ", validate_email,
keep=cur.email),
        "gpa": ask(f"GPA    [{cur.gpa}]: ",
validate_gpa, keep=cur.gpa),
        "course": ask(f"Course
[{cur.course}]: ", validate_course,
keep=cur.course),
    }
    repo.update(roll, **changes)
    print("  OK Student updated.")
```

- Each prompt shows the current value in brackets. Pressing Enter keeps it (via the `keep` argument).
- The results go into a dictionary, which `**changes` expands into keyword arguments

for `repo.update`.

- The roll number itself cannot be changed here, since it is the primary key.

`delete_student`

```
def delete_student(repo):
    roll = ask("\nRoll no to delete: ",
validate_roll_no)
    s = repo.get(roll)
    if input(f"  Delete {s.name}
({s.roll_no})? [y/N]: ").strip().lower() ==
"y":
        repo.delete(roll)
        print("  OK Deleted.")
    else:
        print("  Cancelled.")
```

- Finds the student first, so the confirmation can show their name.
- Only the answer `y` (any case) deletes; anything else, including just pressing Enter, cancels. The capital `N` in `[y/N]` signals that "No" is the default.

The `ACTIONS` dictionary

```
ACTIONS = {"1": add_student, "2": view_all,
"3": search_students,
```

```
            "4": update_student, "5":
delete_student}
```

- Maps each menu choice to the function that handles it. Functions can be stored in dictionaries and called later.
- This replaces a long `if/elif` chain and makes adding a new menu option simple.

---

12. Opening the Database

```
def open_repository():
    try:
        return StudentRepository(DB_FILE)
```

- Tries to create the repository (which runs the integrity check). If successful, returns it.

```
    except CorruptDatabaseError as exc:
        print(f"! {exc}")
        if input("Move the damaged file
aside and start fresh? [y/N]:
").strip().lower() != "y":
            return None
        repo =
StudentRepository.__new__(StudentRepository
)
        repo.db_path = DB_FILE
        print(f"  Old file kept as
```

```
{repo.reset_corrupt_database().name}")
        return repo
```

- If the database is corrupt, it explains the problem and asks permission to recover. Answering anything other than `y` returns `None`, and the program ends.
- If the user agrees:
  - `StudentRepository.__new__(StudentRepository)` creates an object **without** running `__init__`. This is necessary because `__init__` would run the integrity check again and fail immediately.
  - It then sets `db_path` by hand and calls `reset_corrupt_database()`, which backs up the bad file and builds a fresh database.
  - The backup file name is shown, and the ready repository is returned.

```
    except StorageError as exc:
        print(f"! Could not open database:
{exc}")
        return None
```

- Any other storage problem (for example a locked file) shows a message and returns `None`.
- `CorruptDatabaseError` is handled first because it is a subclass of `StorageError`.

13. The Main Loop

```
def main():
    repo = open_repository()
    if repo is None:
        return
```

- Opens the database. If that failed (`None`), the program ends immediately.

```
    while True:
        try:
            print(MENU)
            choice = input("Choose an
option: ").strip()
            if choice == "0":
                print("Goodbye!")
                return
```

- The loop shows the menu and reads the user's choice. Option `0` prints a goodbye message and exits.

```
            action = ACTIONS.get(choice)
            if action is None:
                print("  x Invalid option.
Enter a number from 0 to 5.")
                continue
            action(repo)
```

- Looks up the function for the chosen number. `.get()` returns `None` for unknown input instead of crashing.
- For an invalid choice, it prints a message and `continue` jumps back to the start of the loop.
- For a valid choice, it calls the function, passing in the repository.

```
        except StorageError as exc:
            print(f"  x {exc}")
        except KeyboardInterrupt:
            print("\n  Cancelled.")
        except EOFError:
            print("\nGoodbye!")
            return
```

- These handlers wrap the whole loop body, so no action can crash the program:
  - `StorageError` – any database problem (not found, duplicate, locked) prints a friendly message and the menu returns.
  - `KeyboardInterrupt` (Ctrl+C) – cancels the current action and returns to the menu.
  - `EOFError` (Ctrl+D on Linux/macOS, Ctrl+Z then Enter on Windows) – input has ended, so the program exits cleanly.

---

14. Program Entry Point

```
if __name__ == "__main__":
    main()
```

- Runs `main()` only when the file is executed directly (`python student_records.py`), not when it is imported by another script.

Key Concepts Used

Concept	Where it appears
Dataclass	`Student`
Custom exception hierarchy	`ValidationError`, `StorageError` and its children
Regular expressions	`EMAIL_RE`, `ROLL_RE`
Context manager	`_session` handles connect, commit, rollback, and close in one place
Repository pattern	`StudentRepository` keeps all SQL in one class

Concept	Where it appears
Parameterized queries	Every `?` placeholder, preventing SQL injection
Transactions	Commit on success, rollback on error
Dictionary dispatch	`ACTIONS` maps menu choices to functions
`**kwargs` and unpacking	`update(roll, **changes)` and `(*changes.values(), roll_no)`
Input validation loop	`ask()` repeats until the input is valid